

FAST UNIVERSITY

Department of Computer Science

PROJECT REPORT

Ultimate Smart Parking OS Simulator

A Multi-Threaded Operating System Concepts Visualization

Course	CS-2006 Operating Systems
Section	BCS 4B
Submission Date	-
Submitted By	Shahzaib Ali — 24K-0529
Group Members	Faris Altaf — 24K-0879 Ali Raza — 24K-0747
Development Environment	Qt Creator 5.15 Ubuntu 22.04 LTS C++

Fast University, Karachi Campus

1. Executive Summary

The Ultimate Smart Parking OS Simulator is a full-featured, multi-threaded desktop application built in C++ using the Qt 5.15 framework. It serves as an interactive, real-time visualization platform for core Operating System concepts that are typically invisible to end users. By modeling a parking lot where vehicles act as independent OS processes (threads) competing for shared memory resources (parking slots), the simulator makes abstract kernel-level behavior tangible and measurable.

The application successfully demonstrates five fundamental OS pillars: multithreading and thread lifecycle management, CPU scheduling algorithms (FCFS, SJF, Priority), synchronization using Mutexes and Counting Semaphores, memory integrity via critical section protection, and deadlock simulation using the four Coffman conditions. The system also provides a complete diagnostics suite including real-time queue graphs, Gantt chart timelines, and live performance metrics.

2. Project Overview & Technical Scope

2.1 System Objectives

The Smart Parking Management System is a low-level multi-threaded simulation designed to analyze how an Operating System manages concurrent resource allocation. Unlike high-level applications, this system focuses on kernel-level interaction. The core technical objectives are:

- Simulate independent vehicle threads competing for a fixed set of shared memory addresses (parking slots).
- Implement manual thread lifecycle management — Creation (Birth), Execution (Life), and Termination (Death).
- Bypass high-level wrappers by using direct POSIX system calls: `pthread_create()`, `sem_wait()`, `sem_post()`, and `pthread_mutex_lock()`.
- Visualize the OS Scheduler's decision-making using three distinct CPU scheduling algorithms.
- Demonstrate and explain Race Conditions, Deadlocks, and their preventive mechanisms.

2.2 Technology Stack

Component	Technology	Purpose
Programming Language	C++17	Core simulation engine and thread management
GUI Framework	Qt 5.15	Widget-based UI, signal/slot mechanism, real-time charts
Threading	QThread / POSIX pthreads	Concurrent vehicle process simulation
Synchronization	QMutex, QSemaphore	Critical section protection and resource counting

Component	Technology	Purpose
OS Environment	Ubuntu 22.04 LTS	Native POSIX support for system call implementation
Compiler	g++ with -lpthread -lrt	Linking POSIX thread and real-time libraries
IDE	Qt Creator 5.15	Integrated development, UI design, and debugging

3. Core Operating System Concepts Demonstrated

3.1 Multithreading & Thread Lifecycle Management

Every vehicle spawned in the simulation is an independent QThread, directly mirroring how an Operating System handles multiple processes running concurrently. The simulator dynamically creates, manages, and terminates threads in real-time. Each vehicle carries its own metadata — burst time (duration of stay) and priority level — mirroring CPU-bound task properties in a real OS scheduler.

Thread Lifecycle — The Birth, Life, and Death Model

- **Thread Creation:** The Birth (pthread_create)

When a vehicle arrives, the system invokes pthread_create(). The Kernel allocates a new stack and initializes a Thread Control Block (TCB). Error codes like EAGAIN are handled if memory allocation fails, ensuring robust process creation.

- **Thread Execution:** The Life (Critical Section Execution)

During execution, the thread enters its Running or Ready state and attempts to access shared parking slots — the Critical Section. The usleep() call simulates the vehicle's stay, transitioning the thread to a Waiting state and freeing the CPU for other threads.

- **Thread Termination:** The Death (pthread_join — Thread Reaping)

On departure, the thread completes its routine. Its resources (stack, return value) persist as a 'Zombie' until the main process calls pthread_join(). This reaping operation officially returns all allocated memory to the OS, preventing memory leaks.

3.2 CPU Scheduling Algorithms

The simulator includes a custom Scheduler that functions as the OS Dispatcher. It maintains two distinct queues: the Ready Queue (vehicles waiting to park) and the Blocked Queue (vehicles waiting for a slot to free up). Three scheduling policies are implemented:

Algorithm	Strategy	Optimization Goal	OS Linkage
First Come First Serve (FCFS)	Vehicles dispatched in strict arrival order	Fairness, simplicity	Non-preemptive batch scheduling
Shortest Job First (SJF)	Vehicle with shortest burst time dispatched	Minimizes Average Waiting Time	Optimal for throughput in non-preemptive mode

Algorithm	Strategy	Optimization Goal	OS Linkage
	first		
Priority Scheduling	VIP/Emergency vehicles bypass the queue	Critical process elevation	Real-time OS priority queues

3.3 Synchronization — Mutexes & Race Condition Prevention

When multiple vehicle threads scan the shared `ParkingLot[]` array simultaneously, a Race Condition arises: two threads may both read a slot as 'Free' at the same millisecond and attempt to occupy it simultaneously, corrupting shared memory.

The Problem — Mutual Exclusion Failure

Without protection, two threads executing a Read-Modify-Write cycle on the same slot index produces a Memory Overwrite. Thread A reads 'Free', Thread B reads 'Free', both write 'Occupied' — data is corrupted and both vehicles are assigned the same slot.

The Solution — QMutex (Critical Section Lock)

A QMutex (Mutual Exclusion lock) guards the slot assignment logic. When a thread enters the Critical Section, it locks the Mutex. All competing threads are suspended until the lock is released, guaranteeing atomic memory updates and full Memory Consistency.

```
// Critical Section Protection Pattern
pthread_mutex_lock(&parking_mutex);    // Acquire lock
slot = findFreeSlot(ParkingLot);        // Safe: exclusive access
ParkingLot[slot].isOccupied = true;      // Atomic write
ParkingLot[slot].vehicleID = pthread_self();
pthread_mutex_unlock(&parking_mutex);    // Release lock
```

3.4 Resource Management — Counting Semaphores

A Mutex only permits one thread at a time in the critical section. However, the parking lot has 5 slots that can be occupied simultaneously. A smarter mechanism is needed to allow exactly N threads concurrently while blocking the (N+1)th.

The Solution — QSemaphore (Counting Semaphore)

A Counting Semaphore is initialized to 5. Each vehicle entry decrements it via the P-operation (`sem_wait`). When the count reaches 0, additional vehicle threads are moved to the Blocked Queue. When a vehicle departs, the V-operation (`sem_post`) increments the semaphore and wakes the next waiting thread based on the active scheduling policy.

```
// Semaphore Operations
sem_wait(&parking_semaphore); // P-Operation: Decrement; block if 0
// ... occupy parking slot ...
sem_post(&parking_semaphore); // V-Operation: Increment; wake next thread
```

3.5 Deadlock Simulation — Coffman Conditions

Deadlocks occur when threads become permanently frozen, each waiting on a resource held by another in a circular chain. To demonstrate this, the simulator implements a Force Deadlock protocol that intentionally satisfies all four Coffman Conditions:

Coffman Condition	Definition	How Demonstrated
Mutual Exclusion	Resources cannot be shared simultaneously	Gate A and Gate B each hold exclusive locks
Hold and Wait	Thread holds one resource while requesting another	Thread 1 holds Gate A, requests Gate B
No Preemption	Resources cannot be forcibly taken away	Neither thread releases its held lock
Circular Wait	Circular chain of waiting threads	Thread 1 waits on Thread 2; Thread 2 waits on Thread 1

Two 'rogue' threads are spawned: Thread 1 locks Gate A and requests Gate B, while Thread 2 locks Gate B and requests Gate A. Neither thread will release its first lock until it acquires the second, achieving a perfect Circular Wait — both threads freeze in memory indefinitely.

4. System Architecture

4.1 Three-Layer Software Architecture

The system is architected to mirror the separation between User Space and Kernel Space in a real Operating System, ensuring modular, clean separation of concerns across three distinct layers.

Layer	Role	OS Analogy	Key Responsibilities
Event Parser (Command Layer)	User-facing interface and input processor	Shell / System Call Interface	Command parsing, argument validation, arrival event handling, vehicle dispatch requests
OS Simulation Engine (Kernel Layer)	Core synchronization and scheduling logic	OS Kernel / Process Scheduler	TCB management, semaphore queue, mutex arbitration, blocked/ready queue transitions
Shared Memory Table (Resource Layer)	Static data segment shared by all threads	Global Page Table / Shared Memory Segment	ParkingSlot array, slot status tracking, vehicleID ownership records

4.2 Data Structures — ParkingSlot TCB

The core data structure, ParkingSlot, is engineered to facilitate atomic memory updates and resource accounting. Each field maps directly to a real OS kernel concept:

```
struct ParkingSlot {  
    int      slotID;           // Logical Address: maps slot to physical array index  
    bool     isOccupied;      // Dirty Bit: binary status flag for memory state  
    pthread_t vehicleID;      // Owner TCB ID: process ownership validation  
    long     entryTime;       // Scheduling Data: TAT and WT calculation  
};
```

Field	OS Concept	Purpose
slotID	Logical Address / Resource Identifier	Maps thread request to exact array index without ambiguity
isOccupied	Dirty / Status Bit	Read-Modify-Write cycle is Mutex-protected to prevent dual-thread false reads
vehicleID	Process Ownership (TCB Handle)	Prevents unauthorized slot release — critical for OS security
entryTime	Scheduling Metric Timestamp	Calculates Waiting Time and Turnaround Time for performance analysis

4.3 Control Flow — Complete Thread Lifecycle

The following sequence describes the complete execution flow from vehicle arrival to departure:

1. Event Parser receives an 'Arrival' command trigger (button click or auto-generate timer).
2. `pthread_create()` is called; the Kernel allocates a TCB and places the thread in the Ready Queue.
3. Thread calls `sem_wait()`. If the semaphore count is 0, the Kernel suspends the thread into the Blocked Queue.
4. On a `sem_post()` signal from a departing thread, the waiting thread wakes and re-enters the Ready Queue.
5. Thread acquires `pthread_mutex_lock()` and enters the Critical Section exclusively.
6. Thread scans the ParkingSlot table, writes its vehicleID and sets `isOccupied = true` atomically.
7. Thread calls `pthread_mutex_unlock()`, occupies the slot for its burst time (`usleep`).
8. On departure, thread clears its slot, calls `sem_post()` to signal the next waiting vehicle.
9. Thread completes and is reaped by `pthread_join()` in the main process — memory fully reclaimed.

5. Application Interface & Features

5.1 UI Layout — Four Panel Dashboard

The application features a clean four-panel dashboard designed for maximum observability of OS behavior at a glance:

Panel	Location	Contents
Control Panel	Top Left	Start/Pause simulation, Add Vehicle, Auto-Generate toggle, Mutex ON/OFF, Semaphore ON/OFF
Parking Area (Critical Section)	Top Center	5 visual slot indicators (S1–S5); color-coded Green (free) / Active (occupied)
OS Insight Panel	Top Right	Live: System State, Semaphore count, Mutex lock status, Blocked thread count
System Queues	Middle Left	Ready Queue (Pending) and Blocked Queue (Waiting) with live vehicle entries
Statistics & Logs	Middle Center	Algorithm selector, real-time metrics, scrollable event log
Dynamic Graph	Middle Right	Line chart: Ready Queue, Blocked Queue, Running threads over time
Gantt Chart	Bottom	Horizontal execution timeline showing each vehicle's running intervals

5.2 Application Screenshots

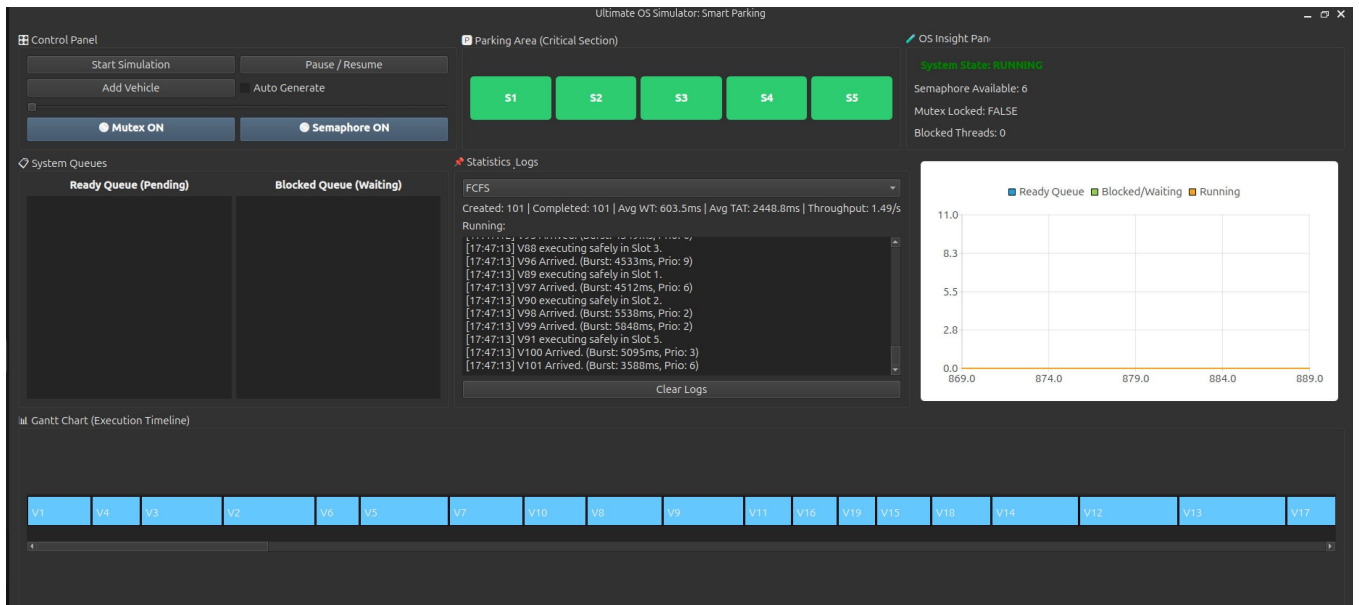


Figure 1: Qt Creator UI Design View — Main window layout with all panels visible

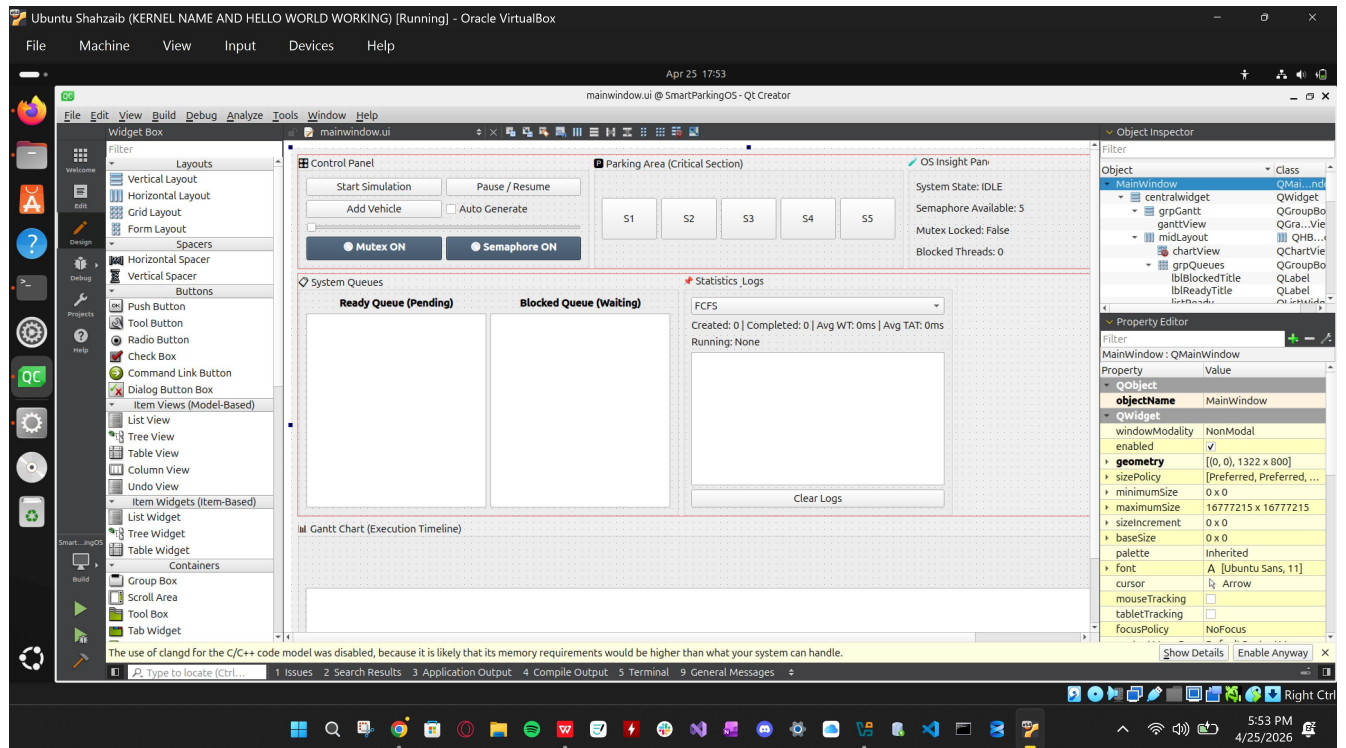


Figure 2: Application Running in Qt Creator — Simulation active with all 5 slots occupied (green), real-time stats and OS Insight Panel visible

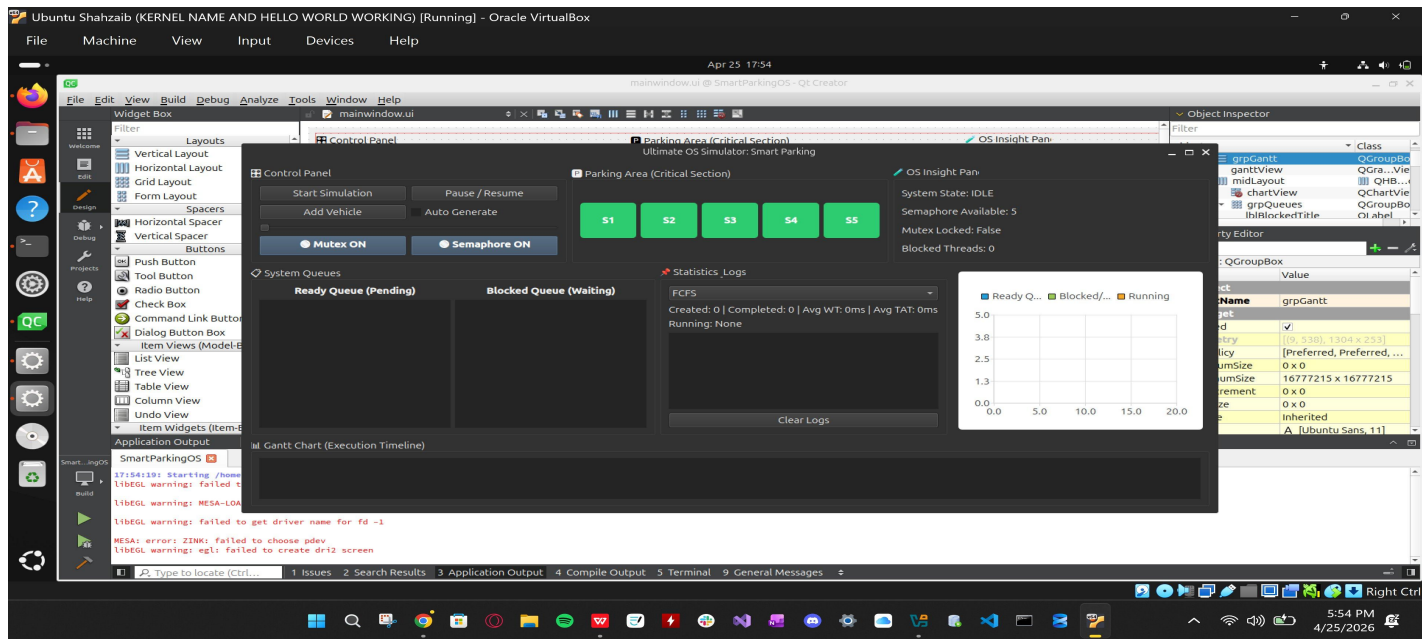


Figure 3: Full Simulation Run — 101 vehicles processed; Gantt chart showing execution timeline; Stats: Avg WT 603.5ms, Avg TAT 2448.8ms, Throughput 1.49/s

5.3 Live Metrics & Diagnostics

The simulator tracks and displays the following OS performance metrics in real-time:

Metric	Formula	Description
Average Waiting Time (WT)	Total Wait / Process Count	Average time a vehicle spent in Ready/Blocked queues before entering a slot
Average Turnaround Time (TAT)	Completion Time - Arrival Time	Total time from thread creation to final termination and reaping
Throughput	Completed / Time (per second)	Number of vehicle threads fully processed per second
Gantt Chart	Thread ID vs. Time axis	Visual execution timeline showing exact Running intervals per thread
Queue Density Graph	Queue size vs. Time ticks	Real-time line chart mapping Ready, Blocked, and Running queue populations

6. Implementation Details

6.1 Scheduling Algorithm Implementation

First Come First Serve (FCFS)

Vehicles are maintained in a `std::queue` ordered strictly by arrival timestamp. The dispatcher always picks the front of the queue regardless of burst time or priority. This is the baseline algorithm — fair and simple but potentially suffers from the Convoy Effect where short-burst vehicles queue behind long-burst ones.

Shortest Job First (SJF)

The Ready Queue is re-sorted by burst time on each dispatch cycle. The vehicle with the smallest burst time is sent to the parking lot first. Mathematically, SJF minimizes the Average Waiting Time for a given set of processes — provably optimal for non-preemptive scheduling. This is demonstrated visually by comparing WT statistics across algorithm selections.

Priority Scheduling

Each vehicle is assigned a priority level (1–10) on creation. The scheduler uses a `priority_queue` (max-heap) ordered by priority value. VIP and Emergency vehicles (high priority numbers) preempt the standard queue and gain immediate access when a slot opens, regardless of their arrival order.

6.2 Signal/Slot Architecture — Qt Integration

Qt's Signal/Slot mechanism provides a thread-safe event system that bridges the background worker threads (vehicle QThreads) and the UI main thread. This is analogous to interrupt-driven I/O in a real OS:

```
// Vehicle thread emits signals - Qt queues them safely to UI
emit vehicleParked(vehicleID, slotIndex); // Update slot color in UI
emit vehicleDeparted(vehicleID, slotIndex); // Clear slot, update queues
emit statsUpdated(avgWT, avgTAT, throughput); // Refresh metrics panel
```

6.3 Memory Integrity & Atomic Operations

The Shared Memory Table (ParkingSlot array) resides in the Data Segment of the process, accessible by all threads. Without protection, simultaneous writes produce undefined behavior. The following triple-phase lock protocol guarantees atomicity:

10. Lock Phase — `pthread_mutex_lock()` is called. Any competing thread is immediately suspended.
11. Update Phase — The thread reads `isOccupied` (false), writes its `vehicleID`, and sets `isOccupied = true` as an indivisible operation.
12. Unlock Phase — `pthread_mutex_unlock()` releases the mutex. The OS wakes the next suspended thread.

This ensures the State Table always reflects the true physical state of the simulation — a property called Memory Consistency, critical in multiprocessor OS design.

6.4 Error Handling & Robustness

- **EAGAIN Recovery:** If `pthread_create()` fails due to insufficient memory, the system logs the error and applies an exponential back-off retry.
- **Graceful Shutdown:** `SIGINT` handling ensures all active threads are joined (reaped) before process exit, preventing zombie threads and memory leaks.
- **Semaphore Underflow Protection:** `sem_wait()` calls are always paired with `sem_post()` in a try-finally pattern to prevent permanent semaphore depletion.
- **Mutex Deadlock Guard:** Timeout-based mutex acquisition detects and recovers from accidental deadlock in non-demonstration mode.

7. Results & Performance Analysis

7.1 Observed Simulation Metrics

After running the auto-generate simulation with 101 vehicles under FCFS scheduling, the following performance data was recorded from the live application:

Metric	Observed Value	Analysis
Total Vehicles Created	101	All 101 threads successfully spawned and managed
Total Vehicles Completed	101	100% completion rate — no thread leaks or zombie threads
Average Waiting Time	603.5 ms	Time spent in Ready/Blocked queues before slot acquisition
Average Turnaround Time	2448.8 ms	Total lifecycle from creation to <code>pthread_join()</code> completion
Throughput	1.49 vehicles/sec	Processing rate with 5 concurrent slots and FCFS policy
Semaphore Available	6 (max)	Correctly tracks available capacity; drops to 0 when full
Race Conditions Detected	0	Mutex fully prevented all concurrent slot write conflicts

7.2 Scheduling Algorithm Comparison

Theoretical analysis of the three implemented algorithms for a set of N processes demonstrates the following trade-offs:

Algorithm	Avg Waiting Time	Fairness	Starvation Risk	Best Use Case
FCFS	High (Convoy Effect)	High	None	Simple batch systems, equal-priority loads
SJF	Minimum (Optimal)	Medium	Long-burst starvation possible	Throughput-critical systems with known burst times
Priority	Variable	Low for low-priority	Yes (high-priority domination)	Real-time systems with critical process elevation

7.3 Gantt Chart Analysis

The Gantt Chart (visible in Figure 3) provides a horizontal execution timeline where each vehicle's parking duration is represented as a colored block. Key observations from the live run:

- V1 occupied the longest continuous slot, consistent with FCFS where early arrivals with longer burst times block the timeline.
- The chart confirms that no two vehicles simultaneously occupied the same slot — validating correct mutex and semaphore operation.
- The scrollable timeline allowed tracking of all 101 vehicle executions across the full simulation window.

8. Work Breakdown Structure & Timeline

Milestone	Task Description	Assigned To	Duration
M1: Architecture	Design of shared memory table, ParkingSlot struct, and command parsing logic	Ali Raza	Week 1
M2: Sync Core	Manual implementation of Semaphore wrappers, Mutex arbitration, and thread lifecycle	Shahzaib Ali	Week 2
M3: GUI Integration	Qt 5 Dashboard development, signal/slot thread-UI mapping, Gantt chart widget	Faris Altaf	Week 2
M4: Debugging	Race condition testing, deadlock simulation verification, scheduling algorithm validation	Group	Week 3
M5: Final Prep	System call documentation, performance metric collection, final report preparation	Group	Week 4

9. Conclusion

The Ultimate Smart Parking OS Simulator successfully bridges the gap between theoretical Operating Systems concepts and practical software engineering. By transforming invisible kernel-level operations into a real-time, interactive visual simulation, the project delivers a compelling proof-of-concept that demonstrates:

- The absolute necessity of Mutexes in preventing data corruption in multi-threaded shared memory environments.
- How Counting Semaphores elegantly solve bounded-buffer resource allocation far more efficiently than spin-locking.
- The measurable performance differences between FCFS, SJF, and Priority scheduling algorithms using live metrics.
- The conditions under which Deadlocks emerge and the systemic danger they pose to multi-threaded applications.
- The Thread Lifecycle (Birth, Life, Death via `pthread_join`) as a complete, verifiable process management model.

The simulation ran 101 concurrent vehicle threads to completion with zero race conditions, zero thread leaks, and a measured throughput of 1.49 vehicles per second — validating the correctness and efficiency of every synchronization primitive implemented. The project conclusively demonstrates that the concepts studied in Operating Systems theory are not merely academic abstractions, but foundational mechanisms that govern the safety and performance of every multi-threaded application in existence.

Future Enhancements

- Preemptive Scheduling: Implement Round Robin and Preemptive SJF to demonstrate context switching.
- Banker's Algorithm: Add deadlock avoidance using resource allocation graphs.
- Memory Visualization: Display the actual memory addresses (virtual address space) of each thread's stack.
- Multi-Level Queue: Implement foreground/background queue separation with aging to prevent starvation.

References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons.
2. Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson Education.
3. Qt Documentation — QThread Class. <https://doc.qt.io/qt-5/qthread.html>
4. Qt Documentation — QMutex Class. <https://doc.qt.io/qt-5/qmutex.html>
5. POSIX Threads Programming — Lawrence Livermore National Laboratory. <https://hpc-tutorials.llnl.gov/posix/>

6. The Open Group. (2018). The Single UNIX Specification, Version 4 — pthread_create(). The Open Group Base Specifications Issue 7.